

Linux Routing Table & Routing Cache

Hands-on Study Notes

Topics: Linux Kernel Implementation of Routing Table and Caches • Routing Cache Implementation Overview • Adding a New Entry in the Routing Table Using the **ip** Command

1. Big Picture

When an IP packet has to be sent, the Linux networking stack must determine where the packet should go. The kernel maintains routing information and uses a routing cache to speed up repeated route lookups.

Application / TCP / UDP → IP layer → Routing lookup → Routing Cache → FIB → Output interface / next hop

Cache hit: the requested route is found quickly. **Cache miss:** the kernel performs a FIB lookup and can update the cache.

2. Routing Cache Implementation Overview

The routing cache stores route information that is currently in use or has been recently used. It is organized as a hash table so that the IP layer can quickly locate a cached route.

Important characteristics:

- A single routing cache can be shared when multiple routing tables are configured for policy routing.
- The cache is organized as an array of **rt_hash_bucket** structures.
- Each bucket contains a chain of cached routes and synchronization information.
- A hash is calculated from routing-related fields such as source, destination and TOS.
- If the cache lookup fails, the kernel performs a FIB lookup.

```
IP layer
↓
Calculate route-cache hash
↓
Select rt_hash_bucket
↓
Search rtable chain
■ Hit → use cached route
■ Miss → FIB lookup → create/use route cache entry
```

3. Important Routing Cache Structures

rt_hash_bucket: one bucket in the routing-cache hash table, containing a chain and synchronization information.

rtable: an individual routing-cache entry containing route-specific information such as source, destination, gateway and flags.

dst_entry: common destination information including device, reference count, usage and expiration information, plus input/output operations.

Structure / Field	Purpose
rt_hash_bucket	Hash bucket containing a chain of cached routes
rtable	Represents one routing-cache entry
rt_src	Source address associated with the route
rt_dst	Destination address
rt_gateway	Gateway/next-hop information
rt_key	Key used for route lookup
rt_flags	Route status/characteristics
rt_type	Route type such as unicast or multicast
dst_entry	Common destination information

dev	Network device associated with the destination
lastuse	Records recent use of the route
expires	Expiration information for cache management

4. Routing Cache Hit and Miss

Cache hit: the cached destination entry is reused and usage/reference information is updated.

Cache miss: the kernel performs a slower FIB lookup, builds the route information and updates the routing cache.

```
ip_route_output_key()
```

↓

Check routing cache

■ ■ HIT → use route

■ ■ MISS → ip_route_output_slow() → FIB lookup → cache entry

5. Managing the Routing Cache

Route-cache entries are retained so recently used routes can be reused. Timers and usage information help remove entries that are no longer useful.

1. Packet needs a route
2. Cache is checked
3. If found, cached route is reused
4. Usage/reference information is updated
5. If not found, FIB lookup is performed
6. Resulting route information can be placed in the cache

6. Forwarding Information Base (FIB)

The FIB is the kernel's internal routing structure used when the required route is not already available in the routing cache.

The implementation uses structures such as **fib_table**, **fn_hash**, **fn_zone**, **fib_node** and **fib_info**.

```
fib_table
↓
fn_hash
■ ■ fn_zone[0] → default route
■ ■ fn_zone[1]
■ ■ ...
■ ■ fn_zone[32] → most specific IPv4 prefix
↓
fib_node
↓
fib_info
```

Longest Prefix Match (LPM): the lookup selects the most specific matching destination prefix.

7. Adding a New Routing Table Entry Using ip

A routing table can be modified from user space through the RT Netlink interface. The **ip** command constructs a Netlink request that is delivered to the kernel.

```
User space
↓
ip route add ...
↓
RT Netlink message
↓
inet_rtm_newroute()
↓
fib_new_table()
↓
fn_hash_insert()
↓
New FIB route entry
```

For a route-add request, the Netlink message identifies the operation as **RTM_NEWROUTE**. The kernel dispatches the request to the route-creation handler.

8. Important Route Message Information

struct rtmsg carries address family, destination prefix length, source prefix length, TOS, routing-table identifier, protocol, scope, route type and route flags.

struct kern_rta contains route attributes such as destination, source, input interface, output interface, gateway and preferred source address.

9. fn_hash_insert()

The route insertion process creates the appropriate FIB structures for the new entry.

```
fn_hash_insert()
■ ■ Get route parameters
■ ■ Find/create appropriate zone
■ ■ Generate hash key using destination + zone mask
■ ■ Create fib_info
■ ■ Locate bucket/chain
■ ■ Check existing destination/TOS/priority entries
■ ■ Create and insert fib_node if required
```

If the corresponding zone does not exist, a new zone is created. The route is then inserted into the appropriate hash chain.

10. Creating Zones and Longest Prefix Matching

The zone is associated with the destination prefix length. The organization can be viewed from a default route at prefix length 0 through increasingly specific prefixes up to 32 bits for IPv4.

- 0 bits → default route
- 8 bits → /8
- 16 bits → /16
- 24 bits → /24
- 32 bits → /32 (host-specific)

This organization supports Longest Prefix Matching: the most specific matching prefix is selected.

11. Practical Linux Commands

Display the routing table:

```
ip route
```

Display routes explicitly:

```
ip route show
```

Display IP addresses:

```
ip addr
```

Display network interfaces:

```
ip link
```

Add a connected route:

```
sudo ip route add 10.10.20.0/24 dev eth0
```

Add a route through a gateway:

```
sudo ip route add 10.10.20.0/24 via 192.168.1.1 dev eth0
```

Ask the kernel which route it would use:

```
ip route get 10.10.20.50
```

Delete a route:

```
sudo ip route del 10.10.20.0/24
```

12. Complete Hands-on Flow

```
USER SPACE
↓
ip route add ...
↓
RT Netlink
↓
inet_rtm_newroute()
↓
fib_new_table()
↓
fn_hash_insert()
↓
FIB entry
↓
Future packet lookup
■ Cache hit → reuse route
■ Cache miss → FIB lookup → update/use cache
```

13. Quick Revision

Question	Answer
Why use a routing cache?	To speed up repeated route lookups.
What happens on a cache miss?	The kernel performs a FIB lookup.
What is rt_hash_bucket?	A bucket in the routing-cache hash table.
What is rtable?	A routing-cache entry.
What is FIB?	The kernel's internal routing information structure used for route lookup.
What does ip route add do?	Requests a new routing entry through RT Netlink.
Which handler processes RTM_NEWROUTE?	inet_rtm_newroute().

What does <code>fn_hash_insert()</code> do?	Inserts the route into the FIB hash structure.
What is LPM?	Longest Prefix Matching; selecting the most specific matching prefix.

Note: These notes describe the Linux routing-cache/FIB implementation discussed in the study material. Modern Linux kernels have evolved significantly, so kernel-internal structures and routing-cache behavior may differ from this implementation.